

PENJELASAN KODE PROGRAM

1. Pada file graph.py

- Class Node

```
@dataclass
class Node:
    """Simpul (titik) dalam graf: lokasi geografis"""
    id: str
    name: str
    lat: float
    lng: float
    node_type: str # 'start' | 'hospital'

    def __hash__(self):
        return hash(self.id)

    def __eq__(self, other):
        if isinstance(other, Node):
            return self.id == other.id
        return False
```

Class Node digunakan untuk merepresentasikan simpul (vertex) pada graf yang menggambarkan suatu lokasi geografis dalam sistem. Setiap objek Node memiliki atribut id sebagai identitas unik, name sebagai nama lokasi, lat dan lng sebagai koordinat geografis, serta node_type yang menunjukkan jenis lokasi, yaitu lokasi awal pengguna (start) atau rumah sakit (hospital). Class ini menggunakan dekorator @dataclass untuk menyederhanakan pembuatan objek dan menyediakan method __hash__() serta __eq__() yang memungkinkan objek node digunakan dalam struktur data seperti set dan dictionary serta dibandingkan berdasarkan nilai id. Dengan demikian, class Node berfungsi sebagai representasi dasar dari setiap titik lokasi yang akan diproses dalam pencarian rute terpendek.

- Class Edge

```
@dataclass
class Edge:
    """Sisi (jalan) dalam graf: koneksi antar node"""
    to_node: Node
    weight: float # jarak dalam km
```

Class Edge digunakan untuk merepresentasikan sisi (edge) atau hubungan antar node pada graf. Setiap objek Edge menyimpan informasi mengenai node tujuan melalui atribut to_node dan bobot hubungan melalui atribut weight. Dalam implementasi sistem ini, bobot digunakan untuk menyatakan jarak antar lokasi dalam satuan kilometer. Class ini berperan sebagai penghubung antar node sehingga struktur jaringan jalan dapat direpresentasikan secara lengkap dan digunakan oleh algoritma Dijkstra untuk menghitung total jarak perjalanan yang paling optimal.

- Class Graph

```

@dataclass
class Graph:
    """Graf berbobot tidak berarah - adjacency list"""
    nodes: Dict[str, Node] = field(default_factory=dict)
    adjacency_list: Dict[str, List[Edge]] = field(default_factory=dict)

    def add_node(self, node: Node):
        self.nodes[node.id] = node
        if node.id not in self.adjacency_list:
            self.adjacency_list[node.id] = []

    def add_edge(self, from_id: str, to_id: str, weight: float, bidirectional: bool = True):
        if from_id not in self.nodes or to_id not in self.nodes:
            raise ValueError(f"Node tidak ditemukan: {from_id} atau {to_id}")

        self.adjacency_list[from_id].append(Edge(self.nodes[to_id], weight))
        if bidirectional:
            self.adjacency_list[to_id].append(Edge(self.nodes[from_id], weight))

    def get_neighbors(self, node_id: str) -> List[Edge]:
        return self.adjacency_list.get(node_id, [])

    def get_starts(self) -> List[Node]:
        return [n for n in self.nodes.values() if n.node_type == 'start']

    def get_hospitals(self) -> List[Node]:
        return [n for n in self.nodes.values() if n.node_type == 'hospital']

```

Class Graph merupakan struktur utama yang digunakan untuk menyimpan seluruh data graf dalam sistem. Class ini memiliki atribut nodes yang berfungsi menyimpan semua objek Node dalam bentuk dictionary dan atribut adjacency_list yang menyimpan hubungan antar node menggunakan metode adjacency list. Beberapa method yang disediakan antara lain add_node() untuk menambahkan node baru, add_edge() untuk membuat hubungan antar node, get_neighbors() untuk mengambil daftar node tetangga, serta get_starts() dan get_hospitals() untuk memperoleh node berdasarkan jenisnya. Penggunaan adjacency list membuat penyimpanan graf menjadi lebih efisien karena hanya menyimpan hubungan yang benar-benar ada. Secara keseluruhan, class Graph berfungsi sebagai wadah utama yang mengelola seluruh lokasi dan koneksi jalan yang akan digunakan dalam proses pencarian jalur.

- Class Algoritma Dijkstra

```

class DijkstraAlgorithm:
    """Algoritma Dijkstra untuk shortest path"""

    def find_path(self, graph: Graph, start_id: str, target_id: str) -> Tuple[Optional[List[Node]], float]:
        """Cari jalur terpendek dari start ke target"""
        dist = {nid: float('inf') for nid in graph.nodes}
        dist[start_id] = 0
        prev = {nid: None for nid in graph.nodes}
        pq = [(0, start_id)]
        visited = set()

        while pq:
            d, u = heapq.heappop(pq)

            if u == target_id:
                break
            if u in visited:
                continue
            visited.add(u)

            for edge in graph.get_neighbors(u):
                v, w = edge.to_node.id, edge.weight
                if d + w < dist[v]:
                    dist[v] = d + w
                    prev[v] = u
                    heapq.heappush(pq, (dist[v], v))

```

```

    if dist[target_id] == float('inf'):
        return None, float('inf')

    # Rekonstruksi path
    path = []
    cur = target_id
    while cur is not None:
        path.append(graph.nodes[cur])
        cur = prev[cur]
    path.reverse()

    return path, dist[target_id]

def find_nearest_hospital(self, graph: Graph, start_id: str):
    """Cari RS terdekat dari start_id"""
    hospitals = [nid for nid, n in graph.nodes.items() if n.node_type == 'hospital']

    best, best_dist, best_id = None, float('inf'), None
    for hid in hospitals:
        p, d = self.find_path(graph, start_id, hid)
        if p and d < best_dist:
            best, best_dist, best_id = p, d, hid

    return best, best_dist, best_id

```

Class DijkstraAlgorithm merupakan implementasi algoritma Dijkstra yang digunakan untuk mencari jalur terpendek pada graf berbobot. Class ini menyediakan method `find_path()` yang menghitung rute dengan total jarak minimum dari node awal menuju node tujuan menggunakan mekanisme priority queue (heapq) untuk memilih node dengan jarak terkecil secara efisien. Selain itu, terdapat method `find_nearest_hospital()` yang memanfaatkan `find_path()` untuk menghitung jarak ke seluruh rumah sakit yang tersedia dan memilih rumah sakit dengan jarak paling dekat dari lokasi awal pengguna. Hasil yang diperoleh berupa jalur yang harus dilalui beserta total jaraknya. Dengan demikian, class DijkstraAlgorithm menjadi komponen utama dalam sistem yang bertanggung jawab menentukan rute tercepat dan rumah sakit terdekat berdasarkan data graf yang telah dibangun.

2. Pada file `data_hospital.py`

```

from core import Node, Graph

def init_graph() -> Graph:
    """Inisialisasi graf jaringan jalan Magelang"""
    g = Graph()

```

Bagian ini merupakan fungsi yang bertugas membangun dan mengembalikan objek graf yang akan digunakan oleh sistem. Objek Graph disimpan pada variabel `g` dan akan menjadi wadah untuk menyimpan seluruh node (lokasi) dan edge (hubungan antar lokasi). Seluruh data titik awal, rumah sakit, dan koneksi jalan akan dimasukkan ke dalam objek ini sebelum digunakan oleh algoritma Dijkstra.

```

#TITIK AWAL
starts = [
    Node("A", "Alun-Alun Magelang", -7.4797, 110.2177, "start"),
    Node("B", "Taman Kyai Langgeng", -7.4856, 110.2224, "start"),
    Node("C", "Museum Pemuda", -7.4703, 110.2186, "start"),
    Node("D", "Pasar Rejowinangun", -7.4822, 110.2133, "start"),
    Node("E", "Stasiun Magelang", -7.4769, 110.2227, "start"),
    Node("F", "Terminal Kebonpolo", -7.4811, 110.2267, "start"),
    Node("G", "Taman Bunga", -7.4733, 110.2156, "start"),
    Node("H", "Kantor Pos Magelang", -7.4789, 110.2194, "start"),
]

```

Bagian ini berisi daftar node yang berperan sebagai titik awal atau lokasi pengguna. Setiap node dibuat menggunakan class Node dengan atribut ID, nama lokasi, koordinat latitude, longitude, dan tipe node "start". Lokasi-lokasi tersebut digunakan sebagai representasi posisi awal pengguna ketika sistem melakukan pencarian rumah sakit terdekat. Contoh pada ID "A", nama lokasi Alun-Alun Magelang, koordinat geografis (-7.4797, 110.2177), dan bertipe start.

```
#RUMAH SAKIT
hospitals = [
    Node("RS1", "Prof.Dr. Soerojo", -7.4708, 110.2183, "hospital"),
    Node("RS2", "Magelang Islamic", -7.4722, 110.2194, "hospital"),
    Node("RS3", "Dr.Soedjono Army", -7.4744, 110.2208, "hospital"),
    Node("RS4", "Tidar Regional", -7.4767, 110.2222, "hospital"),
    Node("RS5", "Lestari Raharja", -7.4783, 110.2233, "hospital"),
    Node("RS6", "Gladool", -7.4797, 110.2247, "hospital"),
    Node("RS7", "Harapan Magelang", -7.4811, 110.2261, "hospital"),
    Node("RS8", "Budi Rahayu", -7.4825, 110.2275, "hospital"),
    Node("RS9", "Merah Putih", -7.4842, 110.2289, "hospital"),
    Node("RS10", "Muntilan Regional", -7.5803, 110.2936, "hospital"),
    Node("RS11", "Aisyiyah Muntilan", -7.5817, 110.2950, "hospital"),
    Node("RS12", "N-21 Gemilang", -7.5831, 110.2964, "hospital"),
    Node("RS13", "Padma Lalita", -7.5844, 110.2978, "hospital"),
]

for n in starts + hospitals:
    g.add_node(n)
```

Bagian ini berisi daftar node yang merepresentasikan rumah sakit sebagai tujuan pencarian. Setiap rumah sakit dibuat menggunakan class Node dengan tipe "hospital". Data yang disimpan meliputi ID rumah sakit, nama rumah sakit, serta koordinat geografisnya. Node-node rumah sakit ini akan menjadi tujuan yang diperiksa oleh algoritma Dijkstra untuk menentukan rumah sakit dengan jarak terpendek dari lokasi pengguna. Contoh pada ID "RS1", Node tersebut merepresentasikan Rumah Sakit Prof. Dr. Soerojo.

Setelah seluruh node titik awal dan rumah sakit dibuat, dimasukkan seluruh node tersebut ke dalam objek graf. Operator + digunakan untuk menggabungkan list starts dan hospitals, kemudian dilakukan iterasi menggunakan for. Setiap node akan ditambahkan ke dalam graf melalui method add_node(). Dengan langkah ini, seluruh lokasi telah terdaftar dalam struktur data graf dan siap untuk dihubungkan menggunakan edge.

```
#EDGE
edges = [
    # Jaringan Titik Awal
    ("A","B",0.9),("A","C",0.7),("A","D",1.1),("A","H",0.5),
    ("B","E",0.8),("B","F",1.2),("B","H",0.6),
    ("C","D",1.3),("C","G",0.4),("C","H",0.8),
    ("D","G",1.5),("E","F",0.7),("E","H",0.4),
    ("F","G",1.8),("G","H",1.0),
    # Titik Awal ke RS
    ("A","RS1",0.8),("A","RS2",1.2),("A","RS3",1.5),("A","RS4",2.1),
    ("B","RS4",0.9),("B","RS5",1.3),("B","RS6",1.8),
    ("C","RS1",0.5),("C","RS2",0.7),
    ("D","RS3",1.1),("D","RS4",1.4),
    ("E","RS4",0.6),("E","RS5",0.9),("E","RS6",1.2),
    ("F","RS6",0.8),("F","RS7",1.1),("F","RS8",1.5),
    ("G","RS2",0.9),("G","RS3",1.2),
    ("H","RS4",0.7),("H","RS5",1.0),
    # Antar RS
    ("RS1","RS2",0.5),("RS2","RS3",0.6),("RS3","RS4",0.7),
    ("RS4","RS5",0.4),("RS5","RS6",0.5),("RS6","RS7",0.6),
    ("RS7","RS8",0.7),("RS8","RS9",0.8),
    # Ke Muntilan
    ("RS9","RS10",12.5),("RS10","RS11",0.8),
    ("RS11","RS12",0.9),("RS12","RS13",1.0),
]
```

Bagian ini berisi seluruh hubungan atau koneksi antar node dalam graf. Setiap data edge terdiri dari tiga elemen yaitu node asal, node tujuan, dan bobot jarak dalam kilometer. Edge digunakan untuk menggambarkan jaringan jalan yang dapat dilalui dari satu lokasi ke lokasi lain. Bobot yang diberikan akan digunakan oleh algoritma Dijkstra untuk menghitung jalur terpendek. Contohnya pada ("A", "B", 0.9), artinya terdapat hubungan antara node A dan node B dengan jarak 0,9 km.

```

for f, t, w in edges:
    g.add_edge(f, t, w, True)

return g

```

Bagian ini digunakan untuk memasukkan seluruh data edge ke dalam graf. Setiap elemen pada list edges akan dipecah menjadi tiga variabel, yaitu f (from), t (to), dan w (weight). Kemudian method `add_edge()` dipanggil untuk membuat hubungan antar node. Parameter `True` menunjukkan bahwa graf bersifat dua arah (bidirectional), sehingga apabila terdapat hubungan A ke B maka secara otomatis juga dibuat hubungan B ke A dengan bobot yang sama.

3. Pada file `app.py`

```

from flask import Flask, render_template, jsonify, request
import requests
from core import DijkstraAlgorithm
from data import init_graph

app = Flask(__name__)

```

Bagian ini digunakan untuk mengimpor seluruh library dan modul yang dibutuhkan sistem. Library Flask digunakan untuk membangun web server dan menyediakan API, requests digunakan untuk mengakses layanan eksternal OSRM (Open Source Routing Machine), sedangkan `DijkstraAlgorithm` dan `init_graph()` diimpor dari modul yang telah dibuat sebelumnya untuk melakukan pencarian jalur terpendek dan memuat data graf.

```

print(" 🔄 Memuat data...")
graph = init_graph()
dijkstra = DijkstraAlgorithm()
total_edges = sum(len(e) for e in graph.adjacency_list.values()) // 2
print(f" ✅ {len(graph.nodes)} node, {total_edges} edge")

```

Bagian ini dijalankan saat aplikasi pertama kali dijalankan. Fungsi `init_graph()` digunakan untuk membangun graf yang berisi titik awal, rumah sakit, dan hubungan antar lokasi. Selanjutnya dibuat objek `DijkstraAlgorithm` yang akan digunakan untuk melakukan perhitungan jalur terpendek. Variabel `total_edges` digunakan untuk menghitung jumlah edge pada graf dan ditampilkan pada terminal sebagai informasi bahwa data berhasil dimuat.

```

def osrm_dist(lat1, lon1, lat2, lon2):
    """Jarak real-time via OSRM API"""
    url = f"http://router.project-osrm.org/route/v1/driving/{lon1},{lat1};{lon2},{lat2}?overview=false"
    try:
        r = requests.get(url, timeout=10).json()
        if 'routes' in r and r['routes']:
            return {
                'distance_km': round(r['routes'][0]['distance'] / 1000, 2),
                'duration_min': round(r['routes'][0]['duration'] / 60, 1),
                'status': 'success'
            }
    except Exception as e:
        print(f"OSRM Error: {e}")
    return {'distance_km': None, 'duration_min': None, 'status': 'error'}

```

Fungsi ini digunakan untuk memperoleh jarak dan estimasi waktu tempuh secara real-time menggunakan layanan OSRM API. Sistem mengirimkan koordinat asal dan tujuan ke server OSRM, kemudian menerima data rute kendaraan berupa jarak dan durasi perjalanan. Jika proses berhasil, fungsi mengembalikan data jarak

dalam kilometer dan durasi dalam menit. Jika terjadi kesalahan, fungsi akan mengembalikan status error sehingga sistem tetap dapat berjalan tanpa mengalami crash.

```
@app.route('/')
def index():
    return render_template('index.html')
```

Endpoint ini digunakan untuk menampilkan halaman utama aplikasi. Ketika pengguna membuka alamat website, Flask akan memuat file index.html yang berisi antarmuka sistem seperti peta, pilihan lokasi, dan hasil pencarian rumah sakit terdekat.

```
@app.route('/api/debug')
def debug():
    first = list(graph.nodes.values())[0]
    return jsonify({
        'status': 'ok',
        'nodes': len(graph.nodes),
        'edges': total_edges,
        'sample': {'id': first.id, 'name': first.name, 'type': first.node_type},
    })
```

Endpoint ini digunakan untuk keperluan pengujian sistem. API akan menampilkan informasi dasar mengenai graf yang sedang digunakan, seperti jumlah node, jumlah edge, dan contoh data node pertama. Endpoint ini membantu pengembang memastikan bahwa data graf berhasil dimuat dengan benar sebelum sistem digunakan.

```
@app.route('/api/nodes')
def get_nodes():
    return jsonify([
        {'id': n.id, 'name': n.name, 'type': n.node_type,
         'latitude': n.lat, 'longitude': n.lng}
        for n in graph.nodes.values()])
```

Endpoint ini digunakan untuk mengambil seluruh data node yang ada pada graf. Data yang dikirimkan berupa ID node, nama lokasi, tipe lokasi, serta koordinat latitude dan longitude. Informasi ini biasanya digunakan oleh frontend untuk menampilkan marker lokasi pada peta digital.

```
@app.route('/api/edges')
def get_edges():
    edges, seen = [], set()
    for fid, el in graph.adjacency_list.items():
        for e in el:
            tid = e.to_node.id
            k = tuple(sorted([fid, tid]))
            if k not in seen:
                seen.add(k)
                edges.append({'from_id': fid, 'to_id': tid, 'weight': e.weight})
    return jsonify(edges)
```

Endpoint ini digunakan untuk mengambil seluruh hubungan antar node dalam graf. Sistem melakukan iterasi terhadap adjacency list dan menghindari duplikasi edge menggunakan variabel seen. Data yang dikembalikan berupa node asal, node tujuan, dan bobot jarak. Endpoint ini biasanya digunakan untuk menggambar garis koneksi antar lokasi pada peta atau visualisasi graf.

```

@app.route('/api/nearest-hospital')
def api_nearest():
    s = request.args.get('start')
    if not s or s not in graph.nodes:
        return jsonify({'error': 'Titik awal tidak valid'}), 400

    path, dist, rid = dijkstra.find_nearest_hospital(graph, s)
    if not path:
        return jsonify({'error': 'Tidak ada RS terjangkau'}), 404

    return jsonify({
        'path': [n.id for n in path],
        'path_details': [{ 'id': n.id, 'name': n.name, 'latitude': n.lat, 'longitude': n.lng, 'type': n.node_type } for n in path],
        'distance': dist,
        'hospital_id': rid,
        'hospital_name': graph.nodes[rid].name,
    })

```

Endpoint ini digunakan untuk mencari rumah sakit terdekat dari suatu titik awal yang dipilih pengguna. Sistem menerima parameter start, kemudian menjalankan method `find_nearest_hospital()` dari algoritma Dijkstra. Hasil yang dikembalikan meliputi jalur yang harus dilalui, total jarak perjalanan, ID rumah sakit, dan nama rumah sakit tujuan. Jika titik awal tidak valid atau tidak ditemukan jalur menuju rumah sakit, sistem akan mengirimkan pesan kesalahan.

```

@app.route('/api/shortest-path')
def api_path():
    s, e = request.args.get('start'), request.args.get('end')
    if not s or not e:
        return jsonify({'error': 'Pilih titik awal dan tujuan'}), 400
    if s not in graph.nodes or e not in graph.nodes:
        return jsonify({'error': 'ID tidak valid'}), 400

    path, dist = dijkstra.find_path(graph, s, e)
    if not path:
        return jsonify({'error': 'Jalur tidak ditemukan'}), 404

    return jsonify({
        'path': [n.id for n in path],
        'path_details': [{ 'id': n.id, 'name': n.name, 'latitude': n.lat, 'longitude': n.lng, 'type': n.node_type } for n in path],
        'distance': dist,
    })

```

Endpoint ini digunakan untuk mencari jalur terpendek antara dua node tertentu. Pengguna harus memberikan parameter start dan end sebagai titik awal dan tujuan. Sistem kemudian menjalankan method `find_path()` untuk menghitung rute dengan jarak minimum. Hasil yang dikembalikan berupa urutan node yang dilalui beserta total jaraknya. Endpoint ini memungkinkan pengguna melihat rute spesifik antara dua lokasi dalam graf.

```

@app.route('/api/compare-distances')
def api_compare():
    s = request.args.get('start')
    if not s or s not in graph.nodes:
        return jsonify({'error': 'Titik awal tidak valid'}), 400

    hs = graph.get_hospitals()
    res = []
    for h in hs:
        p, d = dijkstra.find_path(graph, s, h.id)
        if p:
            res.append({
                'rs_id': h.id, 'rs_name': h.name, 'distance': d,
                'path': [n.id for n in p],
                'path_details': [{ 'id': n.id, 'name': n.name, 'latitude': n.lat, 'longitude': n.lng, 'type': n.node_type } for n in p],
            })

    res.sort(key=lambda x: x['distance'])
    sn = graph.nodes[s]

    return jsonify({
        'start_id': s, 'start_name': sn.name,
        'start_lat': sn.lat, 'start_lng': sn.lng,
        'total_rs': len(res), 'results': res,
    })

```

Endpoint ini digunakan untuk membandingkan jarak dari satu titik awal ke seluruh rumah sakit yang tersedia dalam graf. Sistem menghitung jalur menuju setiap rumah sakit menggunakan algoritma Dijkstra, kemudian menyimpan hasilnya dalam sebuah daftar. Setelah seluruh perhitungan selesai, hasil diurutkan berdasarkan jarak terkecil hingga terbesar. Endpoint ini berguna untuk menampilkan daftar rumah sakit

beserta jaraknya sehingga pengguna dapat membandingkan seluruh alternatif yang tersedia.

```
@app.route('/api/osrm-distance')
def api_osrm():
    try:
        lat1 = float(request.args.get('lat1'))
        lon1 = float(request.args.get('lon1'))
        lat2 = float(request.args.get('lat2'))
        lon2 = float(request.args.get('lon2'))
    except (TypeError, ValueError):
        return jsonify({'error': 'Koordinat tidak valid'}), 400

    return jsonify(osrm_dist(lat1, lon1, lat2, lon2))
```

Endpoint ini digunakan untuk menghitung jarak dan estimasi waktu perjalanan berdasarkan kondisi jaringan jalan sebenarnya menggunakan OSRM API. Sistem menerima koordinat asal dan tujuan dari frontend, kemudian memanggil fungsi `osrm_dist()`. Hasil yang dikembalikan berupa jarak tempuh dalam kilometer dan estimasi waktu perjalanan dalam menit. Endpoint ini melengkapi perhitungan graf statis dengan informasi rute yang lebih realistis.

```
if __name__ == '__main__':
    print("\n🚀 http://localhost:5000\n")
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Bagian ini merupakan titik awal eksekusi program. Ketika file dijalankan secara langsung, Flask akan mengaktifkan web server pada alamat `http://localhost:5000`. Parameter `debug=True` memungkinkan sistem menampilkan informasi kesalahan secara detail selama proses pengembangan, sedangkan `host='0.0.0.0'` memungkinkan aplikasi diakses dari perangkat lain yang berada dalam jaringan yang sama.

4. Pada file main.py

```
1 #!/usr/bin/env python3
2 """Entry point - Jalankan server Flask"""
3
4 from app import app
5
6 if __name__ == '__main__':
7     print("\n📁 Sistem Rute RS Terdekat - Magelang")
8     print("🌐 http://localhost:5000\n")
9     app.run(debug=True, host='0.0.0.0', port=5000)
10
```

File `main.py` berfungsi sebagai entry point atau titik awal eksekusi aplikasi. Pada file ini, objek aplikasi Flask (`app`) diimpor dari file `app.py` kemudian dijalankan menggunakan method `app.run()`. Sebelum server dijalankan, sistem menampilkan informasi nama aplikasi dan alamat akses pada terminal untuk memudahkan pengguna mengetahui lokasi server yang aktif. Pengaturan `debug=True` digunakan untuk mempermudah proses pengembangan dengan menampilkan detail kesalahan secara otomatis, sedangkan `host='0.0.0.0'` dan `port=5000` mengatur agar aplikasi dapat diakses melalui port 5000 baik dari komputer lokal maupun perangkat lain yang berada pada jaringan yang sama. Dengan demikian, file ini berperan sebagai penghubung yang menjalankan seluruh komponen sistem sehingga aplikasi web dapat digunakan oleh pengguna.